

系统开发工具基础 · 实验报告

第 3 周: Packaging and Shipping Code · 智能体编程 · 不止于代码 · Python 与 PyTorch

姓 名	徐宜安	学 号	25060012033
授课教师	周小伟	上课日期	2026 年 9 月 7 日
Git 仓库	GitHub 课程仓库	报告日期	2026 年 9 月 8 日

1 课上练习

1.1 q09 · 从源码构建并在干净环境安装 Wheel (Packaging and Shipping Code)

题目要求

在 q09 中创建一个最小 Python 命令行包，并完成以下操作：

- 创建 `src/greetlab/__init__.py`、`cli.py` 和 `pyproject.toml`，包名中使用自己的学号；
- 运行 `python -m build` 生成 wheel；
- 新建干净虚拟环境，只从生成的 wheel 安装；
- 切换到 q09 之外运行 `sdt-greet --name < 学号 >`，确认输出正确。

实验过程：首先按 `src` 布局创建 `greetlab` 包，`cli.py` 使用 `argparse` 接收必填的 `--name` 参数，`pyproject.toml` 中将项目名称写为 `greetlab-25060012033`，并把 `sdt-greet` 入口指向 `greetlab.cli:main`。两个文件的内容如下。

```
1 [build-system]
2 requires = ["setuptools>=68"]
3 build-backend = "setuptools.build_meta"
4
5 [project]
6 name = "greetlab-25060012033"
7 version = "0.1.0"
8
9 [project.scripts]
10 sdt-greet = "greetlab.cli:main"
```

代码 1 `pyproject.toml`: 构建配置与命令行入口

```
1 import argparse
2
3 def main():
```

```

4     p = argparse.ArgumentParser()
5     p.add_argument("--name", required=True)
6     a = p.parse_args()
7     print(f"Hello, {a.name}!")

```

代码 2 cli.py: 输出姓名问候语

随后在 q09 中执行 `python -m build`。构建工具先创建隔离环境并安装 `setuptools`，最后同时生成源码包和 `wheel`，开始与完成位置分别见图 1(a)、(b)。接着退出 q09，在其上一级目录建立并激活 `test_env`，使用 `pip install` 安装 `dist` 中刚生成的 `wheel`，见图 1(c)。

```

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课上 (main)
$ mkdir -p q09/src/greetlab

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课上 (main)
$ cd q09

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课上/q09 (main)
$ touch src/greetlab/__init__.py

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课上/q09 (main)
$ python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for sdist...

```

(a) 创建包目录并开始构建

```

adding 'greetlab/cli.py'
adding 'greetlab_25060012033-0.1.0.dist-info/METADATA'
adding 'greetlab_25060012033-0.1.0.dist-info/WHEEL'
adding 'greetlab_25060012033-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25060012033-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25060012033-0.1.0.dist-info/RECORD'
removing build\bdist.win-amd64\wheel
Successfully built greetlab_25060012033-0.1.0.tar.gz and greetlab_25060012033-0.1.0-py3-none-any.whl

```

(b) 源码包和 wheel 构建成功

```

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课上/q09 (main)
$ cd ..

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课上 (main)
$ python -m venv test_env

xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课上 (main)
$ source test_env/Scripts/activate
(test_env)
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课上 (main)
$ pip install ./q09/dist/greetlab_25060012033-0.1.0-py3-none-any.whl
Processing d:\desktop\系统开发工具基础\第三周\课上\q09\dist\greetlab_25060012033-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25060012033
Successfully installed greetlab-25060012033-0.1.0

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(test_env)
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课上 (main)
$ sdt-greet --name 25060012033
Hello, 25060012033!
(test_env)
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课上 (main)
$ deactivate

```

(c) 在 q09 外创建干净环境、安装 wheel 并调用命令

图 1 q09 从源码构建到干净环境安装验证的过程

实验结果： wheel 安装后，在 q09 之外运行 `sdt-greet --name 25060012033`，终端输出 `Hello, 25060012033!`。命令行入口能够脱离源码目录运行，说明构建产物和入口配

置均有效。

1.2 q10 · 让编程智能体进入可验证的修复循环（智能体编程）

题目要求

复制 q09 为 q10，使用编程智能体完成一次测试驱动修复：

- 添加失败测试，要求姓名只含空白字符时 main 以 `SystemExit(2)` 结束；
- 先运行测试确认失败，再向智能体说明目标、约束和测试命令，要求其修改实现并运行测试；
- 人工检查 diff，撤销无关修改并再次运行测试；
- 在 `ai_log.md` 中用不超过 5 行记录核心提示、智能体改动和人工验证。

实验过程：先复制 q09，并在 `tests/test_cli.py` 中加入正常姓名和空白姓名两个测试。原来的 main 会直接打印传入内容，因此第一次运行 `pytest` 时正常姓名测试通过，空白姓名测试报出 `DID NOT RAISE SystemExit`，完整失败信息见图 2。

```
python3 -m pytest -v
collected 2 items

tests/test_cli.py::test_main_normal PASSED
tests/test_cli.py::test_main_blank_name FAILED

===== FAILURES =====
test_main_blank_name

monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x000001F727962B90>

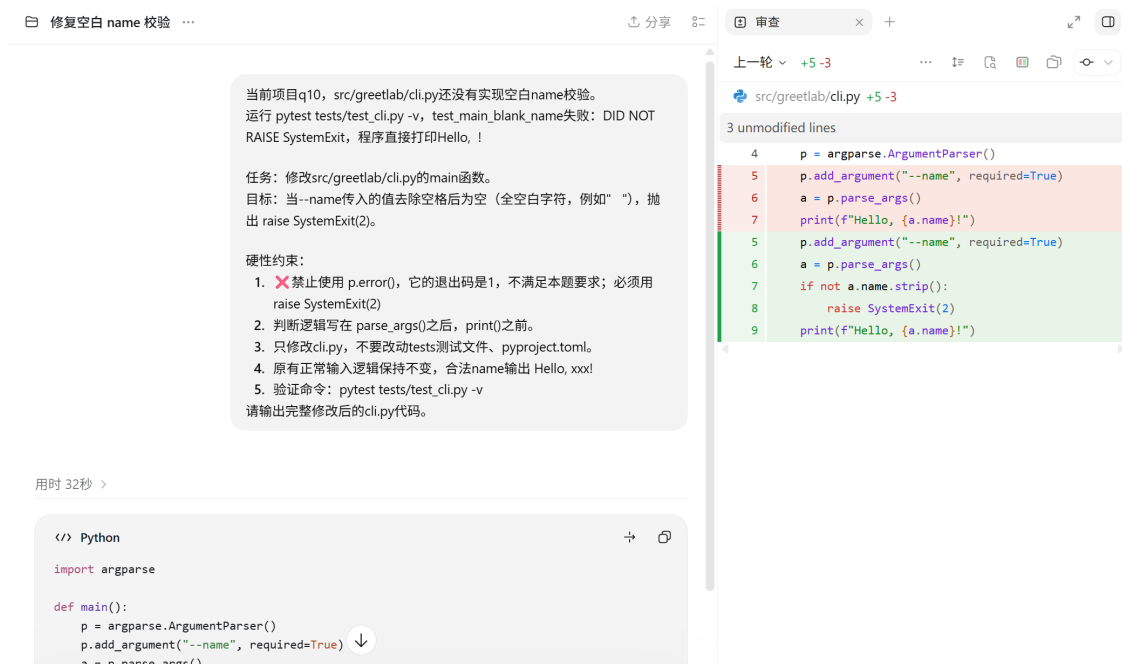
def test_main_blank_name(monkeypatch):
    # name只有空格、空白字符，预期SystemExit(2)
    monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", " "])
    > with pytest.raises(SystemExit) as excinfo:
E       Failed: DID NOT RAISE SystemExit

tests/test_cli.py:13: Failed
----- Captured stdout call -----
Hello,  !

===== short test summary info =====
FAILED tests/test_cli.py::test_main_blank_name - Failed: DID NOT RAISE SystemExit
===== 1 failed, 1 passed in 0.16s =====
```

图 2 q10 加入空白姓名测试后的首次失败结果

确认测试确实能暴露问题后，把失败现象、目标和限制一并交给编程智能体：只允许修改 `src/greetlab/cli.py`，禁止使用退出码不符合要求的 `p.error()`，并给出 `pytest tests/test_cli.py -v` 作为验证命令。智能体增加了 `name.strip()` 检查，修改内容见图 3(a)；随后由智能体运行指定测试，两个用例均通过，见图 3(b)。



(a) 向智能体说明失败现象、修改范围和验证命令

我会只检查并修改 `src/greetlab/cli.py` , 随后按指定命令运行测试验证。

编辑了文件运行了命令

校验已放在 `parse_args()` 之后、`print()` 之前, 并明确使用 `raise SystemExit(2)` ; 现在运行指定测试。

已运行 `pytest tests/test_cli.py -v`

Shell

```
$ pytest tests/test_cli.py -v
plugins: anyio-4.12.0
collecting ... collected 2 items

tests/test_cli.py::test_main_normal PASSED [ 50%]
tests/test_cli.py::test_main_blank_name PASSED [100%]

===== 2 passed in 0.02s =====
```

✓ 成功

(b) 智能体修改后执行 pytest, 两个测试通过

图 3 q10 编程智能体完成修改并运行测试

智能体修改后的实现和用于验证的测试如下。正常姓名仍输出问候语; 姓名去除空白后为空时, 程序在输出前抛出 `SystemExit(2)`。

```
1 import argparse
2
3 def main():
4     p = argparse.ArgumentParser()
5     p.add_argument("--name", required=True)
6     a = p.parse_args()
7     if not a.name.strip():
```

```

8         raise SystemExit(2)
9     print(f"Hello, {a.name}!")

```

代码 3 cli.py: 增加空白姓名校验

```

1  import pytest
2  from greetlab.cli import main
3
4  def test_main_normal(monkeypatch, capsys):
5      monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", "25060012033"])
6      main()
7      out, _ = capsys.readouterr()
8      assert out.strip() == "Hello, 25060012033!"
9
10 def test_main_blank_name(monkeypatch):
11     # name只有空格、空白字符, 预期SystemExit(2)
12     monkeypatch.setattr("sys.argv", ["sdt-greet", "--name", ""])
13     with pytest.raises(SystemExit) as excinfo:
14         main()
15     assert excinfo.value.code == 2

```

代码 4 test_cli.py: 正常姓名与空白姓名测试

最后人工运行 `git diff`, 确认只有 `cli.py` 中与空白校验有关的修改, 再执行一次相同的 `pytest` 命令。终端显示两个测试均通过, 耗时 0.02 s, 见图 4。

```

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第三周/课上/q10 (main)
$ git diff
warning: in the working copy of '第三周/课上/q10/src/greetlab/cli.py', LF will be replaced by CRLF the next time Git touches it
diff --git "a/\347\254\254\344\270\211\345\221\250\350\257\276\344\270\212\q10/src/greetlab/cli.py" "b/\347\254\254\344\270\211\345\221\250\350\257\276\344\270\212\q10/src/greetlab/cli.py"
index 8a6e81d..df1d82f 100644
--- "a/\347\254\254\344\270\211\345\221\250\350\257\276\344\270\212\q10/src/greetlab/cli.py"
+++ "b/\347\254\254\344\270\211\345\221\250\350\257\276\344\270\212\q10/src/greetlab/cli.py"
@@ -4,5 +4,6 @@ def main():
     p = argparse.ArgumentParser()
     p.add_argument("--name", required=True)
     a = p.parse_args()
-    print(f"Hello, {a.name}!")
+    if not a.name.strip():
+        raise SystemExit(2)
+    print(f"Hello, {a.name}!")
\ No newline at end of file

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第三周/课上/q10 (main)
$ pytest tests/test_cli.py -v
===== test session starts =====
platform win32 -- Python 3.11.8, pytest-9.1.1, pluggy-1.6.0 -- D:\Software\python.exe
cachedir: .pytest_cache
rootdir: D:\desktop\系统开发工具基础\第三周\课上\q10
configfile: pyproject.toml
plugins: anyio-4.12.0
collected 2 items

tests/test_cli.py::test_main_normal PASSED [ 50%]
tests/test_cli.py::test_main_blank_name PASSED [100%]

===== 2 passed in 0.02s =====

```

图 4 人工检查 diff 并再次运行测试

修复过程记录见代码 5。

1. 提示: `name`全空白时`raise SystemExit(2)`, 禁止使用`p.error`, 用`pytest`校验

- 2 2. 初始状态：新增测试用例后 `test_main_blank_name` 失败，不会抛出 `SystemExit`
- 3 3. 智能体改动：参数解析完成后增加空白校验，`raise SystemExit(2)`
- 4 4. 人工 diff 检查：未发现无关修改，`pytest` 两个用例全部通过。

代码 5 ai_log.md：智能体修改与人工验证记录

实验结果：空白姓名现在会在打印前以状态码 2 退出，正常姓名的原有行为不变。

1.3 q11 · 把“无法处理”的协作材料改成可执行信息（不止于代码）

题目要求

围绕“空姓名仍输出问候语”这一缺陷，在 `communication.md` 中完成以下改写：

- Issue 包含环境、复现命令、期望结果和实际结果，未知信息写“待确认”；
- 提交信息标题使用祈使语气，正文说明问题和解决方案；
- 评审意见指出具体行为、风险和建议动作，并标注 Blocking、Suggestion 或 Nit；
- 全文控制在 400 字以内，保持专业、简洁。

实验过程：根据已知事实整理环境、复现命令和程序行为，未知版本统一标为“待确认”；提交信息说明修改内容，评审意见用 Blocking 标出需要处理的问题。完整文件见代码 6。

```
1 # Issue: 空白姓名被当作有效输入
2
3 环境: Windows (版本待确认); Shell、Python、greetlab 版本: 待确认。
4
5 复现步骤:
6
7 1. 运行 `sdt-greet --name " "`。
8 2. 检查输出和退出状态。
9
10 期望: 不输出问候语，以状态码 2 退出。
11
12 实际: 输出 `Hello,  !`, 以状态码 0 退出。
13
14 # 提交信息
15
16 标题: `拒绝空白姓名参数输入`
17
18 正文: `--name` 原先接受空白字符串，使无效问候语仍以成功状态结束。解析参数后检查 `name.strip()`，结果为空时抛出 `SystemExit(2)`。
19
20 # 评审意见
21
22 **Blocking:** `main()` 解析参数后直接输出，空白姓名会被当作成功请求，调用脚本可能误判。请在 `print` 前拒绝空白输入，以状态码 2 退出并保留对应测试。
```

代码 6 communication.md: Issue、提交信息与评审意见

1.4 q12 · 修复一个可复现的线性回归训练循环 (Python 与 PyTorch)

题目要求

补全 q12/train.py 并完成一次 PyTorch CPU 训练:

- 在训练循环中正确调用 `zero_grad`、`backward` 和 `step`;
- 训练后调用 `model.eval()`, 并在 `torch.no_grad()` 中计算最终损失;
- 打印最终损失、`weight` 和 `bias`, 最终损失小于 0.001;
- 不得直接把 `weight` 和 `bias` 赋值为 3 和 -1。

实验过程: 使用题目给定的随机种子生成 101 个输入点, 目标值满足 $y = 3x - 1$ 。模型采用单输入、单输出的 `nn.Linear`, 损失函数为均方误差, 优化器为学习率 0.1 的 SGD。每轮根据当前预测计算损失, 清空旧梯度后完成反向传播和参数更新, 共训练 200 轮。训练结束后切换到评估模式, 并关闭梯度记录重新计算最终损失。

```
1  import torch
2  from torch import nn
3
4  torch.manual_seed(20260907)
5  x = torch.linspace(-1, 1, 101).reshape(-1, 1)
6  y = 3 * x - 1
7  model = nn.Linear(1, 1)
8  loss_fn = nn.MSELoss()
9  opt = torch.optim.SGD(model.parameters(), lr=0.1)
10
11 for _ in range(200):
12     pred = model(x)
13     loss = loss_fn(pred, y)
14     opt.zero_grad()
15     loss.backward()
16     opt.step()
17
18 model.eval()
19 with torch.no_grad():
20     final_loss = loss_fn(model(x), y)
21     print(f"final loss: {final_loss.item():.6f}")
22     print(f"weight: {model.weight.item():.6f}")
23     print(f"bias: {model.bias.item():.6f}")
```

代码 7 train.py: 线性回归训练与评估

运行 `python train.py` 后, 程序输出最终损失、`weight` 和 `bias`, 结果见图 5。


```
D:\desktop\系统开发工具基础>python -u "d:\desktop\系统开发工具基础\第三周\课上\q12\train.py"
final loss: 0.000000
weight: 2.999997
bias: -1.000000
```

图 5 q12 PyTorch 线性回归的最终训练结果

实验结果：最终损失为 0.000000，weight 为 2.999997，bias 为 -1.000000，拟合结果与目标直线 $y = 3x - 1$ 一致。

2 课后练习

参考资料记录 [Missing Semester: Packaging and Shipping Code](#)、[PyTorch 教程：学习基础知识](#)。

实例 1：比较虚拟环境激活前后的环境。先记录系统 Python、VIRTUAL_ENV 和 PATH 的前几项，创建并激活 .venv 后再次记录，再用 diff 比较两个文件。

```
1 python -m venv .venv
2 source .venv/Scripts/activate
3 diff -u before.txt after.txt
4 type deactivate | head -n 8
5 deactivate
```

代码 8 创建虚拟环境并比较环境变化

结果见图 6。激活后 Python 和 VIRTUAL_ENV 均指向新建的 .venv，虚拟环境的 Scripts 目录也被放到 PATH 前面；deactivate 实际上是负责恢复原环境变量的 Shell 函数。

```
type deactivate | head -n 8
deactivate
--- before.txt  2026-09-07 16:25:16.600327500 +0800
+++ after.txt   2026-09-07 16:25:37.577938900 +0800
@@ -1,6 +1,6 @@
-python=/d/Software/python
-VIRTUAL_ENV=None
+python=\desktop\系统开发工具基础\第三周\课后\packaging\env-diff\.venv\Scripts/python
+VIRTUAL_ENV=D:\desktop\系统开发工具基础\第三周\课后\packaging\env-diff\.venv
PATH head:
+D
+\desktop\系统开发工具基础\第三周\课后\packaging\env-diff\.venv\Scripts
 /c/Users/xya/bin
-/mingw64/bin
-/usr/local/bin
deactivate is a function
deactivate ()
{
    if [ -n "${_OLD_VIRTUAL_PATH:-}" ]; then
        PATH="${_OLD_VIRTUAL_PATH:-}";
        export PATH;
        unset _OLD_VIRTUAL_PATH;
    fi;
```

图 6 虚拟环境激活前后的路径与环境变量变化

实例 2：检查 wheel 的内部结构。使用 Python 自带的 zipfile 模块列出并解压 q09 生成的 wheel，随后读取 METADATA 和 entry_points.txt。

```
1 python -m zipfile -l ../.../课上/q09/dist/greetlab_25060012033-0.1.0-py3-  
  none-any.whl  
2 python -m zipfile -e ../.../课上/q09/dist/greetlab_25060012033-0.1.0-py3-  
  none-any.whl wheel-files  
3 find wheel-files -type f  
4 cat wheel-files/greetlab_25060012033-0.1.0.dist-info/METADATA  
5 cat wheel-files/greetlab_25060012033-0.1.0.dist-info/entry_points.txt
```

代码 9 列出并解压 wheel

图 7 中可以看到源码、元数据和 RECORD 等文件；项目名为 greetlab-25060012033，版本为 0.1.0，命令行入口仍指向 greetlab.cli:main。

```
cat wheel-files/greetlab_25060012033-0.1.0.dist-info/METADATA  
cat wheel-files/greetlab_25060012033-0.1.0.dist-info/entry_points.txt  
File Name                                     Modified                                     Size  
greetlab/__init__.py                         2026-09-06 19:12:10                       0  
greetlab/cli.py                             2026-09-06 19:01:06                      168  
greetlab_25060012033-0.1.0.dist-info/METADATA 2026-09-06 19:13:16                       67  
greetlab_25060012033-0.1.0.dist-info/WHEEL   2026-09-06 19:13:16                       91  
greetlab_25060012033-0.1.0.dist-info/entry_points.txt 2026-09-06 19:13:16          48  
greetlab_25060012033-0.1.0.dist-info/top_level.txt 2026-09-06 19:13:16           9  
greetlab_25060012033-0.1.0.dist-info/RECORD   2026-09-06 19:13:16          600  
wheel-files/greetlab/cli.py  
wheel-files/greetlab/__init__.py  
wheel-files/greetlab_25060012033-0.1.0.dist-info/entry_points.txt  
wheel-files/greetlab_25060012033-0.1.0.dist-info/METADATA  
wheel-files/greetlab_25060012033-0.1.0.dist-info/RECORD  
wheel-files/greetlab_25060012033-0.1.0.dist-info/top_level.txt  
wheel-files/greetlab_25060012033-0.1.0.dist-info/WHEEL  
Metadata-Version: 2.4  
Name: greetlab-25060012033  
Version: 0.1.0  
[console_scripts]  
sdt-greet = greetlab.cli:main
```

图 7 wheel 文件列表、项目元数据与命令行入口

实例 3：使用环境变量配置程序。程序通过 os.getenv 读取运行模式、端口和调试开关，未设置时使用开发环境默认值。

```
1 import os  
2  
3 mode = os.getenv("APP_MODE", "development")  
4 port = int(os.getenv("APP_PORT", "8000"))  
5 debug = os.getenv("DEBUG", "false").lower() == "true"  
6  
7 print(f"mode={mode}")  
8 print(f"port={port}")  
9 print(f"debug={debug}")
```

代码 10 从环境变量读取运行配置

直接运行时得到 development、8000 和 False；在命令前临时设置变量后，同一份代码输出 production、9000 和 True，见图 8。

```
Code Config_demo.py
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第三周/课后/packaging/config-demo (main)
$ python config_demo.py
mode=development
port=8000
debug=False

xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第三周/课后/packaging/config-demo (main)
$ APP_MODE=production APP_PORT=9000 DEBUG=true python config_demo.py
mode=production
port=9000
debug=True
```

图 8 默认配置与环境变量覆盖后的结果

实例 4：判断版本是否满足依赖约束。为同一组版本分别应用最低版本、范围和兼容版本三种约束，代码见代码 11。

```
1 from packaging.specifiers import SpecifierSet
2
3 versions = ["1.9.0", "2.0.0", "2.1.5", "2.9.0", "3.0.0"]
4 rules = [">>=2.0", ">=2.0,<3.0", "~=2.1.0"]
5
6 for rule in rules:
7     allowed = SpecifierSet(rule)
8     matched = [v for v in versions if v in allowed]
9     print(f"{rule:12} -> {matched}")
```

代码 11 使用 SpecifierSet 判断版本范围

运行结果见图 9。>=2.0 接受 3.0.0，增加 <3.0 后将其排除；~=2.1.0 只匹配兼容的 2.1.5。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第三周/课后/packaging (main)
$ python version_check.py
>=2.0      -> ['2.0.0', '2.1.5', '2.9.0', '3.0.0']
>=2.0,<3.0 -> ['2.0.0', '2.1.5', '2.9.0']
~=2.1.0    -> ['2.1.5']
```

图 9 三种版本约束对应的匹配结果

实例 5：判断 wheel 与当前解释器是否兼容。解析 wheel 文件名中的项目名、版本和兼容标签，再与本机 Python 支持的标签求交集。

```
1 from packaging.tags import sys_tags
2 from packaging.utils import parse_wheel_filename
3
4 wheel = "greetlab_25060012033-0.1.0-py3-none-any.whl"
```

```
5 name, version, build, wheel_tags = parse_wheel_filename(wheel)
6 system_tags = set(sys_tags())
7
8 print("name:", name)
9 print("version:", version)
10 print("wheel tags:", ", ".join(map(str, wheel_tags)))
11 print("compatible:", bool(set(wheel_tags) & system_tags))
12 print("first system tags:")
13
14 for tag in list(sys_tags())[:5]:
15     print(" ", tag)
```

代码 12 解析并比较 wheel 兼容标签

结果见图 10。该 wheel 的标签是 py3-none-any，兼容性判断为 True；本机优先标签则包含 cp311-cp311-win_amd64 等 Windows 平台标签。



```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课后/packaging (main)
$ python wheel_tags.py
name: greetlab-25060012033
version: 0.1.0
wheel tags: py3-none-any
compatible: True
first system tags:
cp311-cp311-win_amd64
cp311-abi3-win_amd64
cp311-none-win_amd64
cp310-abi3-win_amd64
cp39-abi3-win_amd64
```

图 10 wheel 标签与本机解释器标签的比较

实例 6：完成张量切片、矩阵乘法和拼接。对一个 2×3 张量取后两列，与 3×2 张量相乘，并沿第 0 维拼接两份原张量。

```
1 import torch
2
3 a = torch.arange(1, 7).reshape(2, 3)
4 b = torch.tensor([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
5
6 print("a:")
7 print(a)
8 print("slice:")
9 print(a[:, 1:])
10 print("matmul:")
11 print(a.float() @ b)
12 print("cat shape:", torch.cat([a, a], dim=0).shape)
```

代码 13 张量的切片、矩阵乘法与拼接

图 11 中矩阵乘法得到 2×2 结果，拼接后的形状为 `torch.Size([4, 3])`，各项结果与维度计算一致。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第三周/课后/pytorch (main)
$ python tensor_ops.py
a:
tensor([[1, 2, 3],
        [4, 5, 6]])
slice:
tensor([[2, 3],
        [5, 6]])
matmul:
tensor([[ 4.,  5.],
        [10., 11.]])
cat shape: torch.Size([4, 3])
```

图 11 张量切片、矩阵乘法及拼接结果

实例 7：使用 autograd 计算梯度。令 $y = x^3 + 2x$ ，在 $x = 2$ 处反向传播；清空第一次留下的梯度后，再对 $z = x^2$ 反向传播。

```
1 import torch
2
3 x = torch.tensor(2.0, requires_grad=True)
4
5 y = x**3 + 2 * x
6 y.backward()
7 print("y =", y.item())
8 print("dy/dx =", x.grad.item())
9
10 x.grad.zero_()
11 z = x**2
12 z.backward()
13 print("dz/dx =", x.grad.item())
```

代码 14 两次自动求导与梯度清零

程序依次得到 ($y=12$)、 $dy/dx = 14$ 和 $dz/dx = 4$ ，与手工求导相同，见图 12。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第三周/课后/pytorch (main)
$ python autograd_demo.py
y = 12.0
dy/dx = 14.0
dz/dx = 4.0
```

图 12 autograd 的两次梯度计算结果

实例 8：使用 DataLoader 分批读取数据。将 10 个样本放入 TensorDataset，再用批大小为 4 的 DataLoader 按原顺序迭代。

```
1 import torch
```

```
2 from torch.utils.data import DataLoader, TensorDataset
3
4 features = torch.arange(20, dtype=torch.float32).reshape(10, 2)
5 labels = torch.arange(10)
6 dataset = TensorDataset(features, labels)
7 loader = DataLoader(dataset, batch_size=4, shuffle=False)
8
9 for number, (x, y) in enumerate(loader, start=1):
10     print(f"batch {number}: shape={tuple(x.shape)}, labels={y.tolist()}")
```

代码 15 TensorDataset 与 DataLoader 分批读取

输出共包含三个批次，形状依次为 (4,2)、(4,2) 和 (2,2)，说明最后不足一个完整批次的数据仍被正确保留，见图 13。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第三周/课后/pytorch (main)
$ python dataloader_demo.py
batch 1: shape=(4, 2), labels=[0, 1, 2, 3]
batch 2: shape=(4, 2), labels=[4, 5, 6, 7]
batch 3: shape=(2, 2), labels=[8, 9]
```

图 13 DataLoader 生成的三个数据批次

实例 9：保存并重新加载模型参数。先记录随机初始化模型对固定输入的输出，再保存 state_dict；新建同结构模型并加载参数后，比较两次推理结果。完整过程见代码 16。

```
1 import os
2
3 import torch
4 from torch import nn
5
6 torch.manual_seed(7)
7 model = nn.Sequential(nn.Linear(3, 4), nn.ReLU(), nn.Linear(4, 2))
8 x = torch.ones(1, 3)
9
10 model.eval()
11 with torch.no_grad():
12     before = model(x)
13
14 torch.save(model.state_dict(), "weights.pth")
15
16 loaded = nn.Sequential(nn.Linear(3, 4), nn.ReLU(), nn.Linear(4, 2))
17 loaded.load_state_dict(torch.load("weights.pth", weights_only=True))
18 loaded.eval()
19
20 with torch.no_grad():
21     after = loaded(x)
```

```
22
23 print("before:", before.tolist())
24 print("after: ", after.tolist())
25 print("same:", torch.allclose(before, after))
26 print("file size:", os.path.getsize("weights.pth"), "bytes")
```

代码 16 保存、加载并验证模型参数

图 14 中加载前后的两个输出完全一致，`torch.allclose` 返回 `True`，生成的 `weights.pth` 大小为 2461 字节。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第三周/课后/pytorch (main)
● $ python save_load.py
before: [[-0.2582455575466156, 0.2319227159023285]]
after:  [[-0.2582455575466156, 0.2319227159023285]]
same: True
file size: 2461 bytes
```

图 14 模型参数重新加载前后的输出比较

实例 10：构建一个多层神经网络。使用 `Flatten`、两个全连接层和 `ReLU` 组成顺序模型，并用 5 个 2×2 样本完成一次前向计算。

```
1 import torch
2 from torch import nn
3
4 torch.manual_seed(7)
5
6 model = nn.Sequential(
7     nn.Flatten(),
8     nn.Linear(4, 8),
9     nn.ReLU(),
10    nn.Linear(8, 3),
11 )
12
13 x = torch.randn(5, 2, 2)
14 output = model(x)
15 parameters = sum(p.numel() for p in model.parameters())
16
17 print(model)
18 print("input shape:", tuple(x.shape))
19 print("output shape:", tuple(output.shape))
20 print("parameters:", parameters)
```

代码 17 顺序模型的构建与前向计算

运行结果见图 15。输入形状为 (5,2,2)，展平并经过网络后输出形状为 (5,3)；模型共有 67 个可训练参数。

```
xya@DESKTOP-URDSP1Q MINGW64 /d/desktop/系统开发工具基础/第三周/课后/pytorch (main)
$ python network_demo.py
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=4, out_features=8, bias=True)
  (2): ReLU()
  (3): Linear(in_features=8, out_features=3, bias=True)
)
input shape: (5, 2, 2)
output shape: (5, 3)
parameters: 67
```

图 15 网络结构、输入输出形状与参数数量

实例 11：计算分类概率和交叉熵损失。对两组三分类 logits 应用 Softmax，取最大概率对应的类别，并使用 CrossEntropyLoss 计算预测与标签之间的损失。

```
1 import torch
2 from torch import nn
3
4 logits = torch.tensor([
5     [1.0, 2.0, 4.0],
6     [3.0, 1.0, 0.0],
7 ])
8 labels = torch.tensor([2, 0])
9
10 probabilities = torch.softmax(logits, dim=1)
11 predictions = probabilities.argmax(dim=1)
12 loss = nn.CrossEntropyLoss()(logits, labels)
13
14 print("probabilities:")
15 print(probabilities)
16 print("row sums:", probabilities.sum(dim=1))
17 print("predictions:", predictions.tolist())
18 print("loss:", round(loss.item(), 6))
```

代码 18 Softmax 分类概率与交叉熵损失

如图 16 所示，两组概率之和均为 1，预测类别为 [2,0]，与给定标签一致，交叉熵损失为 0.169846。


```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础/第三周/课后/pytorch (main)
● $ python classification_demo.py
probabilities:
tensor([[0.0420, 0.1142, 0.8438],
        [0.8438, 0.1142, 0.0420]])
row sums: tensor([1.0000, 1.0000])
predictions: [2, 0]
loss: 0.169846
```

图 16 Softmax 概率、分类结果与交叉熵损失

3 解题感悟

这一周的任务感觉平时都有接触过，借此机会更好地巩固了它们。Packaging 这部分平时接触过一点，遇到问题基本就是问问 ai，虚拟环境怎么启动跟操作系统、终端都有关老是搞错路径，关于 wheel 和版本控制之前没怎么接触过，学到很多。

关于智能体编程这部分和我平时用 ai 的方式挺接近，但 q10 要求比较多，先留下失败测试，再限制只能改哪个文件，最后还要自己看 diff 和重新跑测试。平时习惯看 ai 写的差不多能跑就用了，还是得需要注意自己检查、测试。另外也能感觉到生产力的变化，刚进大学还是用些网页 ai，后来接触到 agent 基本就回不去了，这也是这个时代不可逆的潮流吧。

“不止于代码”这部分刚开始感觉内容比较简单，毕竟没有多少代码要写，不过写完 Issue、提交信息和评审意见以后，也能感觉到把问题说清楚确实能省很多事。只写一句“运行不了”基本等于还得让别人从头问一遍，环境、复现命令和实际结果这些信息还是一次写完整比较好。

PyTorch 之前有学过一些，基本的 BP 网包括一些简单的卷积神经网络懂得原理，但是基本不会自己写代码，都是靠智能体编程写代码。深度学习神经网络确实彻底改变了这个世界，我记得之前通识课用的时候训练很慢想用 gpu 训练，结果因为 50 系列的显卡当时没有适配的 cuda，配置各种环境配了几乎大半天，但是训练完还是很有成就感的。

这周内容跨度还是挺大的，后续也会在各种实践中继续运用到它们。

第三周课上实验、课后练习及实验报告均已提交至 Git 仓库，相关提交记录见图 17。

```
xya@DESKTOP-URDSPIQ MINGW64 /d/desktop/系统开发工具基础 (main)
● $ git log --oneline -9
15f3c94 (HEAD -> main, origin/main, origin/HEAD) 第三周实验报告大部分
4e9a64f 课后练习部分
0376124 第三周课上q12任务完成
c29afae 第三周课上q11任务完成
449c269 q10补充修改
7a3f9be 第三周课上q10任务完成
dd57b9e 第三周课上q10智能体修改
c382af2 第三周课上q10复制文件夹并创建测试
f397618 第三周课上q09任务完成
```

图 17 第三周实验及报告的 Git 提交记录